PersonalVoice AI

Complete Architecture & Research Blueprint

Agentic, context-aware, personalized voice generation

Project vision

A secure platform where a user creates an authorized personal voice profile once, then generates situation-appropriate audio from text or goals. The system can retrieve the user's own knowledge, plan content and delivery, synthesize speech, evaluate the result, and correct failures within defined limits.

This document consolidates the project definition, existing-model review, proposed research gap, system architecture, workflows, database and API design, evaluation plan, privacy requirements, technology stack, and implementation roadmap.

Prepared as a design blueprint • 17 October 2026

Contents

- 1. Executive summary and product definition
- 2. Existing systems and research baselines
- 3. Research gap and hypotheses
- 4. Complete system architecture
- 5. End-to-end agentic workflow
- 6. Voice onboarding and persistent identity
- 7. RAG and personal knowledge
- 8. Model selection and benchmark protocol
- 9. Backend services and API design
- 10. Database and storage architecture
- 11. Recommended technology stack
- 12. Privacy, security and responsible use
- 13. Evaluation and experiments
- 14. Development roadmap
- 15. Final architecture decisions and next steps
- 16. Reference links

Scope note: this PDF consolidates the architecture discussion and its technical recommendations. The reference list is provided for verification; model capabilities, licensing, pricing, and access conditions must be rechecked before implementation.

1. Executive summary and product definition

PersonalVoice AI is a personalized audio-generation platform designed around four questions: WHO speaks, WHAT should be said, HOW it should sound, and whether the output is accurate, safe, and good enough to deliver.

Question	System responsibility	Primary component
WHO?	Maintain an authorized, reusable voice identity and profile versions.	Voice registry and model adapter
WHAT?	Use user text or create content grounded in authorized personal documents.	Content agent and optional RAG
HOW?	Plan language, pace, pauses, emphasis, and supported delivery settings.	Context and delivery planners
IS IT GOOD?	Evaluate transcription fidelity, speaker similarity, naturalness, and artifacts.	Quality assurance controller

Supported use cases

- Interview self-introductions and practice recordings.
- Presentations, educational narration, storytelling, podcasts, advertisements and personal messages.
- Multilingual speech where the selected engine has demonstrated support.
- Multiple user-managed voice profiles or named delivery presets.
- Optional RAG-grounded scripts based on a résumé, project reports, notes, or other authorized documents.
- Singing is an advanced, separate track because singing synthesis and voice conversion have different technical requirements from ordinary TTS.

Core principle: do not claim that voice cloning itself is new. The proposed contribution is a reliable end-to-end workflow combining persistent voice identity, context-grounded content, explicit delivery planning, quality evaluation, bounded self-correction, and privacy controls.

2. Existing systems and research baselines

The project should benchmark established commercial and open-source systems. The following are candidate references, not a claim that one model is best for every language, accent, or deployment setting.

System	What to study	Design lesson / caveat
Microsoft Personal Voice	Persistent personal voice workflow, consent and profile identifiers.	Consent and profile lifecycle must be designed from the start; access and capabilities are product-dependent.
Google Chirp 3 Instant Custom Voice	Short-sample custom voice creation and cloud synthesis.	Onboarding and recording quality matter; availability may be restricted.
ElevenLabs Voice Cloning	Different cloning tiers, voice management and commercial workflow.	Quality varies with sample quality, task and language; speech and singing are not interchangeable.
F5-TTS	Open-source flow-matching TTS, fidelity, inference speed and resource use.	Run reproducible tests on our hardware and check license and model terms.
CosyVoice 2	Multilingual, zero-shot and streaming-related capabilities.	Test every target language separately; do not infer multilingual consistency from one score.

System	What to study	Design lesson / caveat
IndexTTS	Voice conditioning and controllable speech attributes.	Validate style controls empirically and verify exact model-version support.

Reference pages are listed in Section 16. Always check the current official documentation and repository terms before selecting a dependency.

3. Research gap and hypotheses

The system should target a testable integration problem, not an unsupported claim that existing systems lack all of these capabilities.

Proposed research gap: Can a persistent personal voice identity combined with context-grounded content planning, explicit delivery controls, and automated multi-dimensional quality evaluation improve the reliability and usefulness of personalized audio across real-world tasks?

Limitations and hypotheses to investigate

Area	Hypothesis / risk to test	Proposed response
Persistence	Voice output may vary between engine or model versions.	Versioned profile records, model adapters and regression tests.
Short or noisy samples	Similarity and pronunciation may degrade with insufficient or noisy speech.	Quality gate, recording guidance and confidence-aware onboarding.
Style control	Natural-language style instructions may not reliably control delivery.	Explicit parameter plans and style adherence tests.
Personal facts	A fluent script may invent résumé or project details.	Scoped RAG, source references and factual validation.
Long scripts	Chunk boundaries and long-form synthesis may create discontinuities.	Chunk planning, boundary handling and transcript checks.
Evaluation	Naturalness does not guarantee correct words or identity.	Independent ASR, speaker similarity and human listening tests.
Operations	Failures and retries may increase cost or latency.	Async jobs, bounded retries, idempotency and observability.
Privacy	Voice artifacts are sensitive and can be misused.	Consent, ownership checks, encryption, deletion and auditability.

4. Complete system architecture

The recommended first production design is a modular monolith plus a background inference worker. This is simpler to build and operate than a large microservice fleet while preserving clear module boundaries.

```
USER
|
v
Next.js / React Voice Studio
|
v
FastAPI API + Authentication + Authorization + Consent + Rate Limits
|
v
LangGraph Orchestrator
|-- Context Planner
|-- Knowledge Agent ----> RAG: PostgreSQL + pgvector
|-- Content Agent ------> LLM + factual validation
|-- Delivery Planner ---> Model capability registry
|
v
Voice Model Gateway
```

```
|-- Local open-source engine
|-- Hosted provider adapter (optional)
|-- Profile registry / encrypted model artifacts
|
v
Audio pipeline: preprocessing -> synthesis -> chunk assembly
|
v
Quality assurance
|-- ASR transcript comparison
|-- Speaker similarity
|-- Naturalness / artifact checks
|-- Style adherence
|
+-- PASS --> private object storage --> user delivery
|
+-- FAIL --> targeted correction --> bounded regeneration

Cross-cutting:
PostgreSQL | private object storage | Redis/job queue | audit logs
monitoring | deletion jobs | model registry | cost/latency metrics
```

Key architectural decision: keep orchestration independent from the speech engine. A VoiceEngine interface should expose supported operations and capabilities. Different engines can implement the same interface only where those operations are actually supported; the gateway must not pretend that all models offer identical profile creation, deletion, language, or style features.

Suggested VoiceEngine interface

```
create_profile(...)
synthesize(text, profile, language, delivery_settings, ...)
get_capabilities()
delete_profile(...)
health_check()
```

5. End-to-end agentic workflow

1. Request and permissions: authenticate the user, validate the requested profile, check ownership and consent, and create a unique generation job.
2. Context planning: identify purpose, audience, duration, language, tone and whether personal documents are needed. Ask a question if a consequential detail is ambiguous.
3. Knowledge retrieval: search only the user's authorized knowledge base and retain source references for factual checking.
4. Script drafting: create spoken-language content; validate factual grounding, length, pronunciation-sensitive terms and repetition. Allow user edits before synthesis.
5. Delivery planning: produce supported settings for pace, pauses, emphasis, tone and language. Clamp settings to the chosen model's actual capability range.
6. Voice generation: resolve the selected profile version, call the model gateway, generate audio, assemble chunks if needed, and store media privately.
7. Evaluation: compare ASR transcript with the script, assess speaker similarity against an authorized reference, and check artifacts and supported style signals.
8. Delivery or correction: deliver passing output; otherwise apply a specific correction and retry at most twice more when useful. Avoid indefinite loops.

What makes this agentic?

Agents should make bounded, observable decisions. LangGraph manages state, routing, checkpoints and recovery. The LLM does not generate the waveform; the speech model does. RAG retrieves evidence. Deterministic code must enforce access control, quotas, validation and deletion instead of delegating those decisions solely to an LLM.

Role	Tools	Decision
Orchestrator	Workflow state, policy checks, job status	Which stage runs next
Context planner	Intent classification, duration estimation	What the request requires
Knowledge agent	Scoped search and retrieval	Which evidence is relevant
Content agent	Script generator and factual validator	What to say and what to correct
Delivery planner	Model capability registry	Which supported delivery settings to use
Evaluation controller	ASR, similarity, artifact and style evaluators	Accept, retry, or request human review

6. Voice onboarding and persistent identity

```
Consent + ownership checks
-> record or upload sample
-> normalize audio format
-> detect speech / silence / clipping / noise
-> assess sample quality
-> call engine-specific profile or conditioning workflow
-> generate a fixed test sentence
-> review quality and similarity
-> activate profile
```

Do not assume every voice engine creates a reusable embedding in the same way. Some use a reference recording, some expose a speaker profile ID, and others may require training or fine-tuning. Store a provider-independent profile record and, only when needed, encrypted engine-specific artifacts.

Recommended voice profile fields

- voice_profile_id, user_id, display_name
- engine_id and engine_model_version
- secure profile-artifact reference (never a public URL)
- language capabilities and supported settings
- status: pending, processing, active, failed, deletion pending, deleted
- consent_record_id and verification metadata
- created_at, updated_at and deleted_at

A short clean sample may be enough for some systems, but requirements differ by provider and model. Follow the chosen engine's current documentation rather than imposing one universal sample duration.

7. RAG and the personal knowledge system

```
INGESTION
PDF / DOCX / TXT -> parse -> chunk -> embed -> index

REQUEST TIME
Question -> authorization filter -> retrieval -> relevant passages
-> grounded script -> source / claim validation
```

RAG is optional. It is not needed when a user simply provides text to read aloud. It becomes useful when a request refers to a résumé, project, personal notes, presentation or other stored knowledge.

Recommended metadata per document chunk

- owner_id / tenant_id, document_id, source location and chunk index
- document version, ingestion timestamp and access policy
- embedding model/version and content hash
- retention state and deletion status

Security requirements: enforce permissions before retrieval; treat uploaded documents as untrusted data rather than instructions; flag unsupported personal claims; preserve source references; and remove indexed chunks when a document is deleted, consistent with documented backup-retention rules.

8. Model selection and benchmark protocol

Start by benchmarking F5-TTS and CosyVoice 2; include IndexTTS if its license, hardware requirements and controls fit. Use a hosted personal-voice service as a reference when access is available. These are candidates, not a predetermined winner.

Criterion	Measure
Speaker similarity	Speaker-embedding cosine similarity plus human listening review.
Content accuracy	Word error rate (WER) from an independent ASR system.
Naturalness	Human MOS-style ratings on a representative subset.
Delivery control	Whether pace, pauses, emphasis and intended style are followed.
Accent robustness	Results on representative consented accent and speaking samples.
Multilingual consistency	Similarity, pronunciation and intelligibility per supported language.
Speed	Time to first audio and total generation time.
Cost	GPU time or provider cost per generated minute.
Reliability	Failure rate, retries, memory usage and long-text performance.
Deployment constraints	License, model weights, hardware, access restrictions and commercial-use terms.

Interpret metrics together. Speaker similarity alone does not prove naturalness or accuracy; WER alone does not prove identity preservation. Record model version, sample, settings and results for every run. Keep evaluation samples separate from any samples used to tune the system.

9. Backend services and API design

Keep version 1 modular. The API handles authentication, validation, orchestration and status; a worker handles expensive synthesis.

Method / endpoint	Purpose
POST /v1/auth/register	Create account
POST /v1/auth/login	Authenticate
POST /v1/voices	Start voice onboarding
GET /v1/voices	List user's profiles

Method / endpoint	Purpose
GET /v1/voices/{id}	Inspect profile status
PATCH /v1/voices/{id}	Update supported metadata or version
DELETE /v1/voices/{id}	Initiate profile deletion
POST /v1/knowledge/documents	Upload a document
GET /v1/knowledge/documents	List accessible documents
DELETE /v1/knowledge/documents/{id}	Delete document and index
POST /v1/generations	Submit generation job
GET /v1/generations/{id}	Poll status and evaluation
GET /v1/generations	List generation history
DELETE /v1/generations/{id}	Delete generated content

Generation jobs should be asynchronous and idempotent. Suggested states: queued → processing → evaluating → completed, or failed. Use a unique idempotency key for submissions, bounded retries for transient failures, and short-lived signed URLs for downloads. Never expose storage paths or provider credentials.

10. Database and storage architecture

```
users
|-- voice_profiles
| |-- voice_versions
| |-- consent_records
|-- documents
| |-- document_chunks (pgvector)
|-- generations
| |-- generation_evaluations
|-- user_preferences
|-- audit_events
|-- deletion_jobs
|-- model_registry
```

Use PostgreSQL for identity, permissions, metadata and generation history. Store audio binaries and large artifacts in private object storage, not ordinary database rows. A profile ID is not authorization: every read, update, synthesis and deletion must verify resource ownership.

11. Recommended technology stack

Layer	Recommended starting choice	Reason
Frontend	Next.js, React, TypeScript, Tailwind CSS	Voice studio, profile manager, script editor and history.
API	Python, FastAPI, Pydantic	Typed APIs and integration with Python ML/audio libraries.
Orchestration	LangGraph	Stateful workflow, checkpoints, bounded retries and routing.
Database / vectors	PostgreSQL + pgvector	Relational metadata and vector search in one data layer.
Prototype vector DB	ChromaDB (optional)	Convenient for early local experiments.
Object storage	Private S3-compatible storage	Scalable audio and document storage.
Audio processing	FFmpeg, torchaudio, librosa, VAD	Normalization, segmentation and preprocessing.
ASR / evaluation	Whisper or suitable ASR; speaker encoder	Transcript fidelity and similarity measurement.
Jobs	Redis + worker queue	Keep expensive synthesis out of API handlers.
Deployment	Docker first, then AWS as requirements become clear	Reproducibility and measured production scaling.
Observability	Structured logs, metrics, tracing and alerts	Debugging, latency and cost monitoring.

12. Privacy, security and responsible use

- Consent: explicit consent and verification appropriate to the workflow.
- Access control: strict ownership checks, private media and short-lived download links.
- Encryption: protect transport, database/object storage, backups and sensitive artifacts.
- Deletion: revoke access promptly, then remove voice artifacts, recordings, indexes and generated media under a documented retention/backup policy.
- Provenance: retain generation metadata and provide synthetic-audio disclosure where appropriate. Investigate watermarking, but do not treat it as a complete defense.
- Abuse prevention: rate limits, consent enforcement, monitoring and safeguards against deceptive impersonation.
- Auditability: log profile creation, consent changes, model version, generation attempts and deletion outcomes without unnecessarily logging sensitive content.
- Human control: let users review scripts and audio before sharing or publication.

For the MVP, restrict custom-voice enrollment to the account holder's own voice and design for authorized, transparent use. Do not facilitate impersonating another person or covert generation for deceptive identity claims.

13. Evaluation and experiments

Compare three systems using the same prompts, consented voices and evaluation conditions. Where possible, hold the underlying voice model constant to isolate the value of the orchestration architecture.

System	Pipeline
Baseline A	Text → voice model → audio.
Baseline B	Context-aware script generation → voice model → audio.
Proposed	Context + optional RAG → delivery planning → voice generation → independent evaluation → bounded correction.

Metrics

- Script factuality and unsupported-claim rate.
- Speaker similarity and word error rate.
- Human-rated naturalness and delivery appropriateness.
- Percentage of generations passing all quality gates.
- First-pass success and quality improvement after retries.
- End-to-end latency, cost per generated minute and failure rate.

Set numerical targets after the pilot benchmark. Any claim of improvement must be supported by repeated runs, defined metrics and uncertainty estimates. A retry that improves quality but triples cost is not automatically an improvement.

14. Development roadmap

Phase	Focus	Deliverable
0	Research and benchmark	Candidate-model comparison, license review, test protocol and architecture decisions.
1	Voice-generation MVP	Record/reference sample → model → text → audio playback.
2	Persistent profiles and backend	Authentication, PostgreSQL, consent, lifecycle and deletion.
3	Content and RAG	Document ingestion, scoped retrieval, grounded script and editing.
4	Agentic orchestration	Observable LangGraph workflow, routing, checkpoints and bounded retries.
5	Evaluation and experiments	Reproducible report for similarity, WER, naturalness, style, latency and cost.
6	Production hardening	Quotas, monitoring, deletion jobs, recovery, security tests and deployment automation.
7	Advanced features	Multilingual consistency, streaming, richer style controls and separate singing module.

Build in this order. Do not start with a large multi-agent system or expensive GPU deployment before validating a speech model. Keep version 1 as a modular monolith plus a background worker.

15. Final architecture decisions and next steps

Recommended version 1

- Product: PersonalVoice AI.
- Architecture: modular monolith plus a background inference worker.
- Orchestration: LangGraph with bounded, observable workflows.
- Knowledge: optional RAG using PostgreSQL + pgvector.

- Voice: benchmark-driven model gateway supporting local and hosted engines.
- Evaluation: separate transcript, speaker-similarity, naturalness and delivery checks.
- Storage: PostgreSQL for metadata; private object storage for audio.
- Deployment: Docker first, AWS after resource requirements are measured.
- Trust: consent, ownership, deletion, provenance and auditability from the start.

Immediate next task: create a model-benchmark specification with candidate model versions, hardware requirements, license checks, test samples, metrics and acceptance criteria. Use those results to lock the voice-engine adapter before implementing the remaining modules.

The defensible contribution is the measured improvement of the end-to-end personalized audio workflow—not the unsupported claim that voice cloning itself is new.

16. Reference links

F5-TTS research paper
https://arxiv.org/abs/2410.06885

CosyVoice 2 research paper
https://arxiv.org/abs/2412.10117

IndexTTS official repository
https://github.com/index-tts/index-tts

Microsoft Personal Voice documentation
https://learn.microsoft.com/en-us/azure/ai-services/speech-service/personal-voice-create-voice

Google Chirp 3 Instant Custom Voice documentation
https://docs.cloud.google.com/text-to-speech/docs/chirp3-instant-custom-voice

LangGraph documentation
https://langchain-ai.github.io/langgraph/

ElevenLabs voice cloning documentation
https://elevenlabs.io/docs

Research note: product capabilities, access requirements, pricing, model licenses and commercial-use terms can change. Verify the current official source before implementation. This document is an architecture proposal; its hypotheses and recommended features still require empirical validation.